FUNDAMENTAL THEOREM OF SUBSTRUCTURAL PARAMETRICITY WITH PURELY POSITIVE RECURSIVE TYPES

C. B. ABERLÉ, KARL CRARY, CHRIS MARTENS, FRANK PFENNING

Editorial note. Source https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2025.4. Copyright C. B. Aberlé, Karl Crary, Chris Martens and Frank Pfenning, CC BY4.0. Original statements and all printed author proof material are faithfully converted. Theorem9 is the sole counted target; base and mixed-variable fundamental theorems and illustrative consequences are supporting context. Author omissions and termination proviso are retained without supplementation.

Allowed standard prerequisites (editorial).

- Parametric logical predicates, monoids and structural induction. Publisher sections2-6 and references. The actual ordered calculus, algebraic predicate, recursive type restrictions and author proof cases are included, rather than replaced by prerequisite citations.

2 A Minimalist Fragment

We start with a small fragment of the Lambek Calculus [25, 29], extended with parametric polymorphism [43]. This fragment is sufficient to illustrate the main ideas behind our constructions. For the sake of simplicity we choose a Curry-style formulation of typing, concentrating on properties of untyped terms rather than intrinsically typed terms. This allows the same terms to inhabit ordered, linear, and unrestricted types and thereby focus on semantic rather than syntactic issues.

$$
\begin{array}{llll}
\text{Types} & A, B & ::= & \alpha \mid \mathbf{1} \mid A \bullet B \mid A \rightarrowtail B \mid A \twoheadrightarrow B \mid \forall \alpha.\, A \\
\text{Terms} & e & ::= & x \\
& & \mid & () \mid \mathbf{match}\ e\ (() \Rightarrow e') \qquad (\mathbf{1}) \\
& & \mid & (e_1, e_2) \mid \mathbf{match}\ e\ ((x, y) \Rightarrow e') \qquad (A \bullet B) \\
& & \mid & \lambda x.\ e \mid e_1\, e_2 \qquad (A \rightarrowtail B, A \twoheadrightarrow B)
\end{array}
$$

In this fragment, we have $A \bullet B$ (read "A fuse B") which, logically, is a noncommutative conjunction. We have two forms of implication: $A \rightarrowtail B$ (read: "A under B", originally written as $A \setminus B$) which is true if from the hypothesis A placed at the left end of the antecedents we can deduce B, and $A \twoheadrightarrow B$ (read: "B over A", originally written as B/A) which is true if from the hypothesis A placed at the right end of the antecedents we can prove B. Lambek's original notation was suitable for the sequent calculus and its applications in linguistics, but is less readable for natural deduction and functional programming.

Our basic typing judgment has the form $\Delta \mid \Omega \vdash e : A$ where Δ consists of hypotheses α type, and Ω is an ordered context $(x_1 : A_1) \ldots (x_n : A_n)$. We make the standard presuppositions that $\Delta \vdash A$ type and $\Delta \vdash A_i$ type for every $x_i : A_i$ in Ω, and that both type variables and term variables are pairwise distinct. The rules are show in Figure 1.

Date: doi:10.4230/LIPIcs.FSCD.2025.4; CC BY4.0 (official publisher HTML/PDF).

$$\frac{}{\Delta \mid x : A \vdash x : A}\ \mathsf{hyp}$$

$$\frac{}{\Delta \mid \cdot \vdash () : \mathbf{1}}\ \mathbf{1}I \qquad \frac{\Delta \mid \Omega \vdash e : \mathbf{1} \quad \Delta \mid \Omega_L \Omega_R \vdash e' : C}{\Delta \mid \Omega_L \Omega \Omega_R \vdash \mathbf{match}\ e\ (() \Rightarrow e') : A}\ \mathbf{1}E$$

$$\frac{\Delta \mid \Omega\,(x : A) \vdash e : B}{\Delta \mid \Omega \vdash \lambda x.\, e : A \twoheadrightarrow B}\ \twoheadrightarrow I \qquad \frac{\Delta \mid \Omega \vdash e_1 : A \twoheadrightarrow B \quad \Delta \mid \Omega_A \vdash e_2 : A}{\Delta \mid \Omega\,\Omega_A \vdash e_1\, e_2 : B}\ \twoheadrightarrow E$$

$$\frac{\Delta \mid (x : A)\,\Omega \vdash e : B}{\Delta \mid \Omega \vdash \lambda x.\, e : A \rightarrowtail B}\ \rightarrowtail I \qquad \frac{\Delta \mid \Omega \vdash e_1 : A \rightarrowtail B \quad \Delta \mid \Omega_A \vdash e_2 : A}{\Delta \mid \Omega_A\,\Omega \vdash e_1\, e_2 : B}\ \rightarrowtail E$$

$$\frac{\Delta \mid \Omega_A \vdash e_1 : A \quad \Delta \mid \Omega_B \vdash e_2 : B}{\Delta \mid \Omega_A\,\Omega_B \vdash (e_1, e_2) : A \bullet B}\ \bullet I \qquad \frac{\Delta \mid \Omega \vdash e : A \bullet B \quad \Delta \mid \Omega_L\,(x : A)\,(y : B)\,\Omega_R \vdash e' : C}{\Delta \mid \Omega_L\,\Omega\,\Omega_R \vdash \mathbf{match}\ e\ ((x, y) \Rightarrow e') : C}\ \bullet E$$

$$\frac{\Delta, \alpha\,\mathsf{type} \mid \Omega \vdash e : A}{\Delta \mid \Omega \vdash e : \forall \alpha.\, A}\ \forall I \qquad \frac{\Delta \mid \Omega \vdash e : \forall \alpha. A(\alpha) \quad \Delta \vdash B\ \mathsf{type}}{\Delta \mid \Omega \vdash e : A(B)}\ \forall E$$

Figure 1: Ordered Natural Deduction.

Here are a few example judgments that hold or fail. We elide the context $\Delta = (\alpha\,\mathsf{type}, \beta\,\mathsf{type}, \gamma\,\mathsf{type})$.

	$\vdash$	$\lambda x.\, x : \alpha \rightarrowtail \alpha$	
	$\vdash$	$\lambda x.\, x : \alpha \twoheadrightarrow \alpha$	
	$\nvdash$	$\lambda x.\, \lambda y.\, x : \alpha \twoheadrightarrow (\beta \twoheadrightarrow \alpha)$	(no weakening)
	$\nvdash$	$\lambda x.\, (x, x) : \alpha \twoheadrightarrow (\alpha \bullet \alpha)$	(no contraction)
	$\vdash$	$\lambda x.\, \lambda y.\, (x, y) : \alpha \twoheadrightarrow (\beta \twoheadrightarrow (\alpha \bullet \beta))$	
	$\nvdash$	$\lambda x.\, \lambda y.\, (x, y) : \alpha \rightarrowtail (\beta \rightarrowtail (\alpha \bullet \beta))$	(no exchange)
$f : \beta \twoheadrightarrow (\alpha \rightarrowtail \gamma)$	$\vdash$	$\lambda x.\, \lambda y.\, (f\, y)\, x : \alpha \rightarrowtail (\beta \twoheadrightarrow \gamma)$	("associativity")
$g : \alpha \rightarrowtail (\beta \twoheadrightarrow \gamma)$		$\lambda y.\, \lambda x.\, (g\, x)\, y : \beta \twoheadrightarrow (\alpha \rightarrowtail \gamma)$	
$g : (\alpha \bullet \beta) \twoheadrightarrow \gamma$	$\vdash$	$\lambda x.\, \lambda y.\, g\,(x, y) : \alpha \twoheadrightarrow (\beta \twoheadrightarrow \gamma)$	(currying)
$f : \alpha \twoheadrightarrow (\beta \twoheadrightarrow \gamma)$		$\lambda p.\, \mathbf{match}\ p\ ((x, y) \Rightarrow f\, x\, y) : (\alpha \bullet \beta) \twoheadrightarrow \gamma$	(uncurrying)

The strictures of the typing judgment imply that certain types may be uninhabited, or may be inhabited by terms that are extensionally equivalent (i.e., that yield the same outputs for all closed inputs) to a small number of possibilities. To count the number of linear functions, translate $(A \twoheadrightarrow B)^{\text{L}} = (A \rightarrowtail B)^{\text{L}} = A^{\text{L}} \multimap B^{\text{L}}$ and $(A \bullet B)^{\text{L}} = A^{\text{L}} \otimes B^{\text{L}}$ and similarly for unrestricted functions.

Types	Ordered	Linear	Unrestricted
$\alpha \twoheadrightarrow \alpha$	1	1	1
$\alpha \twoheadrightarrow (\alpha \twoheadrightarrow \alpha)$	0	0	2
$\alpha \twoheadrightarrow (\alpha \twoheadrightarrow (\alpha \bullet \alpha))$	1	2	4
$\alpha \twoheadrightarrow (\alpha \rightarrowtail (\alpha \bullet \alpha))$	1	2	4
$\alpha \twoheadrightarrow (\beta \twoheadrightarrow (\beta \bullet \alpha))$	0	1	1
$\alpha \twoheadrightarrow (\beta \twoheadrightarrow (\alpha \bullet \beta))$	1	1	1

Because our intended application language based on adjoint natural deduction [19] is call-by-value, we can give a straightforward big-step operational semantics [22] relating a term to its final value. Because this evaluation does not directly interact with or benefit from substructural properties, we show it without further comment in Figure 2. It has the property of preservation that if $\cdot \vdash e : A$ and $e \hookrightarrow v$ then $\cdot \vdash v : A$. Jang et al. give an account [19] that exploits linearity and other substructural properties, although not the lack of exchange.

$$\frac{}{() \hookrightarrow ()} \qquad \frac{e \hookrightarrow () \quad e' \hookrightarrow v}{\mathbf{match}\ e\ (() \Rightarrow e') \hookrightarrow v}$$

$$\frac{}{\lambda x.\, e \hookrightarrow \lambda x.\, e} \qquad \frac{e_1 \hookrightarrow \lambda x.\, e_1' \quad e_2 \hookrightarrow v_2 \quad [v_2/x]e_1' \hookrightarrow v}{e_1\, e_2 \hookrightarrow v}$$

$$\frac{e_1 \hookrightarrow v_1 \quad e_2 \hookrightarrow v_2}{(e_1, e_2) \hookrightarrow (v_1, v_2)} \qquad \frac{e \hookrightarrow (v_1, v_2) \quad [v_1/x, v_2/y]e' \hookrightarrow v'}{\mathbf{match}\ e\ ((x, y) \Rightarrow e') \hookrightarrow v'}$$

Figure 2: Big-Step Operational Semantics.

3 An Algebraic Logical Predicate

Because of our particular setting, we define two mutually dependent logical predicates: $[\![A]\!]$ for closed terms and $[A]$ for closed values. In addition, the relation is parameterized by elements from an algebraic domain which may have various properties. For the ordered case, it should be a monoid, for the linear case a commutative monoid. However, the rules themselves do not require this for the pure sets of terms. We use $m \cdot n$ for the binary operation on the monoid, and ϵ for its unit.

Ignoring polymorphism for now, we write $m \Vdash e \in [\![A]\!]$ and $m \Vdash v \in [A]$, which is defined by

$$\begin{array}{lcl}
m \Vdash e \in [\![A]\!] & \iff & \exists v.\, e \hookrightarrow v \wedge m \Vdash v \in [A] \\
\\
m \Vdash v \in [\mathbf{1}] & \iff & m = \epsilon \wedge v = (\,) \\
m \Vdash v \in [A \bullet B] & \iff & \exists m_1, m_2.\, m = m_1 \cdot m_2 \wedge v = (v_1, v_2) \\
 & & \quad \wedge\ m_1 \Vdash v_1 \in [A] \wedge m_2 \Vdash v_2 \in [B] \\
m \Vdash v \in [A \twoheadrightarrow B] & \iff & \forall k.\, k \Vdash w \in [A] \Longrightarrow m \cdot k \Vdash v\, w \in [\![B]\!] \\
m \Vdash v \in [A \rightarrowtail B] & \iff & \forall k.\, k \Vdash w \in [A] \Longrightarrow k \cdot m \Vdash v\, w \in [\![B]\!]
\end{array}$$

We can see how the algebraic structure of the monoid tracks information about order if its operation is not commutative.

The key step, as usual in logical predicates of this nature, is the case for universal quantification and type variables. We map type variables α to relations R_B between monoid elements and values in $[B]$ where B is a closed type. We use S to denote this mapping from type variables to sets of values and write it as a superscript on $\Vdash$.

$$\begin{array}{lcl}
m \Vdash^S v \in [\alpha] & \iff & m\ S(\alpha)\ v \\
m \Vdash^S v \in [\forall \alpha.\, A(\alpha)] & \iff & \forall B, R_B.\, m \Vdash^{S, \alpha \mapsto R_B} v \in [A(\alpha)]
\end{array}$$

The mapping S is just passed through identically in the cases of the relation defined above.

We can already verify some interesting properties. As a first example we show that the logical predicates are nonempty.

Example 1.

$$\epsilon \Vdash \lambda x.\, \lambda y.\, (x, y) \in [\![\forall \alpha.\, \alpha \twoheadrightarrow (\alpha \twoheadrightarrow (\alpha \bullet \alpha))]\!]$$

Proof.

Because the λ-term is a value, we need to check

$$\epsilon \Vdash \lambda x.\, \lambda y.\, (x, y) \in [\forall \alpha.\, \alpha \twoheadrightarrow (\alpha \twoheadrightarrow (\alpha \bullet \alpha))]$$

By definition, this is true if for an arbitrary A and relation $m\ R_A\ v$ we have

$$\epsilon \Vdash^{\alpha \mapsto R_A} \lambda x.\, \lambda y.\, (x, y) \in [\alpha \twoheadrightarrow (\alpha \twoheadrightarrow (\alpha \bullet \alpha))]$$

Using the definition of the logical predicate for right implication twice and one intermediate step of evaluation, this holds iff

$$m \cdot k \Vdash^{\alpha \mapsto R_A} (\lambda y.\, (v, y))\ w \in [\![\alpha \bullet \alpha]\!]$$

for all m, k with $m \Vdash^{\alpha \mapsto R_A} v$ and $k \Vdash^{\alpha \mapsto R_A} w$. By evaluation, this is true iff

$$m \cdot k \Vdash^{\alpha \mapsto R_A} (v, w) \in [\alpha \bullet \alpha]$$

Now we can apply the definition of $[A \bullet B]$, splitting $m \cdot k$ into m and k and reducing it to

$$m \Vdash^{\alpha \mapsto R_A} v \in [\alpha] \wedge k \Vdash^{\alpha \mapsto R_A} w \in [\alpha]$$

Both of these hold because, by assumption, $m\ R_A\ v$ and $k\ R_A\ w$. ◀

More interesting, perhaps, is the reverse.

Example 2.

If

$$\epsilon \Vdash e \in [\![\forall \alpha.\, \alpha \twoheadrightarrow (\alpha \twoheadrightarrow (\alpha \bullet \alpha))]\!]$$

then e is extensionally equal to $\lambda x.\, \lambda y.\, (x, y)$, in that for any closed type A and closed terms $v, w : A$, it must be the case that $e\, v\, w \hookrightarrow (v, w)$. In particular, it can not be $\lambda x.\, \lambda y.\, (y, x)$.

Proof.

We choose our monoid to be the free monoid over two generators a and b and we choose an arbitrary closed type A and two elements v and w. Moreoever, we pick R_A relating only $a\ R_A\ v$ and $b\ R_A\ w$.

From the definitions (and skipping over some simple properties regarding evaluation), we obtain

$$a \cdot b \Vdash^{\alpha \mapsto R_A} e\ v\ w \in [\![\alpha \bullet \alpha]\!]$$

By the clauses for $[\![\alpha \bullet \alpha]\!]$, $[\alpha \bullet \alpha]$ and α we conclude that

$$e\ v\ w \hookrightarrow (u_1, u_2)$$

for some values u_1 and u_2 with $a\ R_A\ u_1$ and $b\ R_A\ u_2$. Because the only value related to a is v and the only value related to b is w, we conclude $u_1 = v$ and $u_2 = w$. Therefore

$$e\ v\ w \hookrightarrow (v, w)$$

Since v and w were chosen arbitrarily, we see that e is extensionally equal to $\lambda x.\, \lambda y.\, (x, y)$. ◀

4 The Fundamental Theorem

The fundamental theorem of logical predicates states that every well-typed term is in the predicate. Our relations also include terms that are not well-typed, which can occasionally be useful when one exceeds the limits of static typing.

We need a few standard lemmas, adapted to this case. We only spell out one.

Lemma 3 (Compositionality).
Define R_A such that $k\ R_A\ w$ iff $k \Vdash w \in [A]$. Then $m \Vdash^{S,\alpha\mapsto R_A} v \in [B(\alpha)]$ iff $m \Vdash^S v \in [B(A)]$

Proof.
By induction on $B(\alpha)$. ◀

We would like to prove the fundamental theorem by induction over the structure of the typing derivation. Since our logical relation is defined for closed terms, we need a closing substitution η, i.e., a substitution of closed terms into the typing context of the term e for which we seek to prove the fundamental theorem by induction, such that the application $\eta(e)$ of η to e is a closed term. We therefore extend the definition of the logical relation to substitutions and contexts as follows:

$$
\begin{array}{lcl}
m \Vdash^S (x \mapsto v) \in [x : A] & \iff & m \Vdash^S v \in [A] \\
m \Vdash^S (\eta_1\, \eta_2) \in [\Omega_1\, \Omega_2] & \iff & \exists m_1, m_2.\, m = m_1 \cdot m_2 \wedge m_1 \Vdash^S \eta_1 \in [\Omega_1] \wedge m_2 \Vdash^S \eta_2 \in [\Omega_2] \\
m \Vdash^S (\cdot) \in [\cdot] & \iff & m = \epsilon
\end{array}
$$

Due to the associativity of the monoid operation and concatenation of contexts, this constitutes a valid definition.

Theorem 4 (Fundamental Theorem (purely ordered)).
Assume $\Delta \mid \Omega \vdash e : A$, a mapping S with domain Δ, and closing substitution $m \Vdash^S \eta \in [\Omega]$. Then $m \Vdash^S \eta(e) \in [\![A]\!]$.

Proof.
By induction on the structure of the given typing derivation. We show a few cases.

Case:

$$\frac{}{\Delta \mid x : A \vdash x : A}\ \mathsf{hyp}$$

Then $m \Vdash^S \eta(x) \in [A]$ by assumption and definition, and $m \Vdash^S \eta(x) \in [\![A]\!]$ since $\eta(x)$ is a value.

Case:

$$\frac{\Delta \mid \Omega\,(x : A) \vdash e : B}{\Delta \mid \Omega \vdash \lambda x.\, e : A \twoheadrightarrow B}\ {\twoheadrightarrow}I$$

$m \Vdash^S \eta \in [\Omega]$ Given
$k \Vdash^S v \in [A]$ Assumption (1)
$k \Vdash^S (x \mapsto v) \in [x : A]$ By definition
$m \cdot k \Vdash^S (\eta, x \mapsto v) \in [\Omega\,(x : A)]$ By definition
$m \cdot k \Vdash^S (\eta, x \mapsto v)(e) \in [\![B]\!]$ By ind. hyp.
$m \cdot k \Vdash^S (\eta(\lambda x.\, e))\, v \in [\![B]\!]$ By reverse evaluation, v closed
$m \Vdash^S \eta(\lambda x.\, e) \in [A \twoheadrightarrow B]$ By definition, discharging (1)
$m \Vdash^S \eta(\lambda x.\, e) \in [\![A \twoheadrightarrow B]\!]$ By definition

Case:

$$\frac{\Delta \mid \Omega \vdash e_1 : A \twoheadrightarrow B \quad \Delta \mid \Omega_A \vdash e_2 : A}{\Delta \mid \Omega\,\Omega_A \vdash e_1\, e_2 : B}\ {\twoheadrightarrow}E$$

$m \Vdash^S \eta \in [\Omega\,\Omega_A]$ Given
$m_1 \Vdash^S \eta_1 \in [\Omega]$ and $m_2 \Vdash^S \eta_2 \in [\Omega_A]$
for some m_1, m_2, η_1, and η_2 with $m = m_1 \cdot m_2$ and $\eta = \eta_1\ \eta_2$ By definition

$m_1 \Vdash^S \eta_1(e_1) \in [\![A \twoheadrightarrow B]\!]$ By ind. hyp.
$m_2 \Vdash^S \eta_2(e_2) \in [\![A]\!]$ By ind. hyp.
$\eta_1(e_1) \hookrightarrow v_1$ with $m_1 \Vdash^S v_1 \in [A \twoheadrightarrow B]$ By definition
$\eta_2(e_2) \hookrightarrow v_2$ with $m_2 \Vdash^S v_2 \in [A]$ By definition
$m_1 \cdot m_2 \Vdash^S v_1\, v_2 \in [\![B]\!]$ By definition
$(\eta_1\, \eta_2)(e_1\, e_2) = (\eta_1(e_1))\,(\eta_2(e_2))$ By properties of substitution
$m \Vdash^S \eta(e_1\, e_2) \in [\![B]\!]$ Since $m = m_1 \cdot m_2$ and $\eta = (\eta_1\, \eta_2)$
Case:

$$\frac{\Delta, \alpha\,\mathsf{type} \mid \Omega \vdash e : A}{\Delta \mid \Omega \vdash e : \forall\alpha.\, A}\ \forall I$$

$m \Vdash^S \eta \in [\Omega]$ Given
R_B an arbitrary relation $k\ R_B\ v$ Assumption (1)
$m \Vdash^{S,\alpha\mapsto R_B} \eta \in [\Omega]$ Since α fresh
$m \Vdash^{S,\alpha\mapsto R_B} \eta(e) \in [\![A]\!]$ By ind. hyp.
$m \Vdash^S \eta(e) \in [\![\forall\alpha.\, A]\!]$ By definition, discharging (1)
Case:

$$\frac{\Delta \mid \Omega \vdash e : \forall\alpha.A(\alpha) \quad \Delta \vdash B\ \mathsf{type}}{\Delta \mid \Omega \vdash e : A(B)}\ \forall E$$

$m \Vdash^S \eta \in [\Omega]$ Given
$m \Vdash^S \eta(e) \in [\![\forall\alpha.\, A(\alpha)]\!]$ By ind. hyp.
Define $k\ R_B\ v$ iff $k \Vdash^S v \in [B]$
$m \Vdash^{S,\alpha\mapsto R_B} \eta(e) \in [\![A(\alpha)]\!]$ By definition
$m \Vdash^{S,\alpha\mapsto R_B} v \in [A(\alpha)]$ for $\eta(e) \hookrightarrow v$ By definition
$m \Vdash^S v \in [A(B)]$ By compositionality (Lemma 3)
$m \Vdash^S \eta(e) \in [\![A(B)]\!]$ By definition

◀

Because typing implies that the logical predicate holds, the earlier examples now apply to well-typed terms.

Theorem 5 (Example 2 revisited).
If

$$\cdot \vdash e : \forall\alpha.\, \alpha \twoheadrightarrow (\alpha \twoheadrightarrow (\alpha \bullet \alpha))$$

then e is extensionally equivalent to $\lambda x.\, \lambda y.\, (x, y)$.

Proof.
We just note that

$$\epsilon \Vdash e \in [\![\forall\alpha.\, \alpha \twoheadrightarrow (\alpha \twoheadrightarrow (\alpha \bullet \alpha))]\!]$$

since $(\cdot) \in [\cdot]$ and $(\cdot)e = e$ and the empty mapping S suffices without any free type variables. Then we appeal to the reasoning in Example 2. ◀

5 Unrestricted Functions

We are interested in properties of functions such as list append or list reversal, or higher-order functions such as fold. This requires inductive types, but the functions on them are not used linearly. For example, append has a recursive call in the case of a nonempty list, but none in the case of an empty list. We could introduce a general modality $!A$ for this purpose. A simpler alternative that is sufficient for our situation is to introduce unrestricted function types $A \to B$ (usually coded as $!A \multimap B$ in linear logic or $!A \twoheadrightarrow B$ in ordered logic). This path has been explored previously [37] with different motivations. There, an open logical relation was

defined on the negative monomorphic fragment in order to show the existence of canonical forms, a property that is largely independent of ordered typing.

Adding unrestricted functions is rather straightforward in typing by using two kinds of variables: those that are ordered and those unrestricted. Then, in the logical predicate, unrestricted variables must not use any resources, that is, they are assigned the unit element ϵ of the monoid during the definition.

The generalized judgment has the form $\Delta \mid \Gamma \;;\; \Omega \vdash e : A$ where Γ contains type assignments for variables that can be used in an unrestricted (not linear and not ordered) way. All the previous rules are augmented by propagating Γ from the conclusion to all premises. Because our term language is untyped, no extensions are needed there. Similarly, the rules of our dynamics do not need to change.

$$\frac{}{\Delta \mid \Gamma, x : A \;;\; \cdot \vdash x : A}\ \mathsf{hyp}$$

$$\frac{\Delta \mid \Gamma, x : A \;;\; \Omega \vdash e : B}{\Delta \mid \Gamma \;;\; \Omega \vdash \lambda x.\, e : A \to B}\ {\to}I \qquad \frac{\Delta \mid \Gamma \;;\; \Omega \vdash e_1 : A \to B \quad \Delta \mid \Gamma \;;\; \cdot \vdash e_2 : A}{\Delta \mid \Gamma \;;\; \Omega \vdash e_1\, e_2 : B}\ {\to}E$$

Figure 3: Unrestricted functions.

We extend the logical predicate using arguments not afforded any resources.

$$m \Vdash v \in [A \to B] \iff \forall w.\, \epsilon \Vdash w \in [A] \Longrightarrow m \Vdash v\, w \in [\![B]\!]$$

The fundamental theorem extends in a straightforward way.

Theorem 6 (Fundamental Theorem (mixed ordered/unrestricted)).
Assume $\Delta \mid \Gamma \;;\; \Omega \vdash e : A$, a mapping S with domain Δ, and two closing substitutions $\epsilon \Vdash^S \theta \in [\Gamma]$ and $m \Vdash^S \eta \in [\Omega]$. Then $m \Vdash^S (\theta \;;\; \eta)(e) \in [\![A]\!]$.

Proof.
By induction on the structure of the given typing derivation. ◀

An interesting side effect of these definitions is that if we omit ordered functions but retain pairs we obtain the "usual" formulation of closed logical predicates, including certain consequences of parametricity for the ordinary λ-calculus.

Theorem 7.
If

$$\cdot \vdash e : \forall \alpha.\, \alpha \to (\alpha \to (\alpha \bullet \alpha))$$

then e is extensionally equivalent to one of 4 functions: $\lambda x.\, \lambda y.\, (x, y)$, $\lambda x.\, \lambda y.\, (y, x)$, $\lambda x.\, \lambda y.\, (x, x)$, or $\lambda x.\, \lambda y.\, (y, y)$.

Proof.
By the fundamental theorem, we have

$$\epsilon \Vdash e \in [\![\forall \alpha.\, \alpha \to (\alpha \to (\alpha \bullet \alpha))]\!]$$

We use this for an abitrary closed type A with two arbitary values v, and w and relation R_A with $\epsilon\ R_A\ v$ and $\epsilon\ R_A\ w$. Exploiting the definition, we get

$$\epsilon \Vdash^{\alpha \mapsto R_A} e \in [\![\alpha \to (\alpha \to (\alpha \bullet \alpha))]\!]$$

Using the definition of function twice and skipping over some evaluation and reverse evaluation, we obtain

$$\epsilon \Vdash^{\alpha \mapsto R_A} e\, v\, w \in [\![\alpha \bullet \alpha]\!]$$

This means that $e\,v\,w \hookrightarrow (u_1, u_2)$ with $\epsilon\ R_A\ u_1$ and $\epsilon\ R_A\ u_2$. Because of the definition of R_A there are 4 possibilities for (u_1, u_2), namely (v, w), (w, v), (v, v) and (w, w). This in turn means e is extensionally equal to one of the 4 functions shown. ◀

6 Sums, Twist, and Recursive Types

At this point, we are at a crossroads. Because we would like to prove theorems regarding more complex data structures such as lists, trees, or streams, we could extend the development with general inductive and coinductive types and their recursors. We conjecture that this is possible and leave it to future work. The other path is to work with purely positive types, including recursive ones whose values can be directly observed. In this approach, the definition of the logical predicate is quite easy to extend. It becomes a nested inductive definition: either the type becomes smaller or, once we encounter a purely positive type and recursion is possible, from then on the terms become strictly smaller. In this paper we take the latter approach, which excludes coinductive types such as streams from consideration, but still yields many interesting and intuitive consequences.

We take the opportunity to also round out our language with sums and twist (the symmetric counterpart of fuse). We use a signature defining equirecursive type names that may be arbitrarily mutually recursive. Because such type definitions are otherwise closed, they constitute metavariables in the sense of contextual modal type theory [31]. Each type definition $F[\Delta] = A^+$ (for Δ a context of type variables as in previous sections) must be contractive, that is, its definiens cannot be be another type name. Moreover, A^+ must be purely positive, which is interpreted inductively.

Types	A	::=	$\ldots \mid A \circ B \mid \oplus\{\ell : A_\ell\}_{\ell \in L}$
Purely Positive Types	A^+, B^+	::=	$A^+ \bullet B^+ \mid A^+ \circ B^+ \mid \mathbf{1} \mid \oplus\{\ell : A^+_\ell\}_{\ell \in L} \mid F[\theta]$
Type Definitions	Σ	::=	$F[\Delta] = A^+ \mid (\cdot) \mid \Sigma_1, \Sigma_2$
Type Substitutions	θ	::=	$\alpha \mapsto A^+ \mid (\cdot) \mid \theta_1\,\theta_2$

The language of terms does not change much because type names are equirecursive.

Term	e	::=	$\ldots$
		$\mid$	$k(e) \mid \mathbf{match}\ e\ \{\ell(x_\ell) \Rightarrow e'\}_{\ell \in L} \quad (\oplus\{\ell : A_\ell\})$

We add the type $A \circ B$ ("twist"), symmetric to $A \bullet B$, since encoding it as $B \bullet A$ requires rewriting terms, flipping the order of pairs. For $A \circ B$ it is merely the typechecking that changes. This allows more types to be assigned to the same term. We allow silent unfolding of type definitions, so there are no explicit rules for $F[\theta]$.

$$\frac{\Delta \mid \Gamma \,;\, \Omega_A \vdash e_1 : A \quad \Delta \mid \Gamma \,;\, \Omega_B \vdash e_2 : B}{\Delta \mid \Gamma \,;\, \Omega_B \,\Omega_A \vdash (e_1, e_2) : A \circ B} \circ I$$

$$\frac{\Delta \mid \Gamma \,;\, \Omega \vdash e : A \circ B \quad \Delta \mid \Gamma \,;\, \Omega_L \,(y : B)\,(x : A)\,\Omega_R \vdash e' : C}{\Delta \mid \Gamma \,;\, \Omega_L \,\Omega\, \Omega_R \vdash \mathbf{match}\ e\ ((x, y) \Rightarrow e') : C} \circ E$$

$$\frac{(k \in L) \quad \Delta \mid \Gamma \,;\, \Omega \vdash e : A_k}{\Delta \mid \Gamma \,;\, \Omega \vdash k(e) : \oplus\{\ell : A_\ell\}_{\ell \in L}} \oplus I$$

$$\frac{\Delta \mid \Gamma \,;\, \Omega \vdash e : \oplus\{\ell : A_\ell\}_{\ell \in L} \quad (\Delta \mid \Gamma \,;\, \Omega_L \,(x_\ell : A_\ell)\,\Omega_R \vdash e_\ell : A_\ell) \quad (\forall \ell \in L)}{\Delta \mid \Gamma \,;\, \Omega_L \,\Omega\, \Omega_R \vdash \mathbf{match}\ e\ \{\ell(x_\ell) \Rightarrow e_\ell\}_{\ell \in L} : C} \oplus E$$

Figure 4: Ordered Natural Deduction, Extended.

$$\frac{e \hookrightarrow v}{k(e) \hookrightarrow k(v)} \qquad \frac{e \hookrightarrow k(v) \quad [v/x_k]e_k \hookrightarrow v'}{\mathbf{match}\ e\ \{\ell(x_\ell) \Rightarrow e_\ell\}_{\ell \in L} \hookrightarrow v'}$$

Figure 5: Big-Step Operational Semantics, Extended.

The logical predicate is also extended in a straightforward manner. We assume the signature Σ is fixed and therefore do not carry it explicitly through the definitions.

$$\begin{array}{lcl}
m \Vdash^S v \in [A \circ B] & \iff & \exists m_1, m_2.\ m = m_2 \cdot m_1 \wedge v = (v_1, v_2) \\
 & & \quad \wedge\, m_1 \Vdash^S v_1 \in [A] \wedge m_2 \Vdash^S v_2 \in [B] \\
m \Vdash^S k(v) \in [\oplus\{\ell : A_\ell\}_{\ell \in L}] & \iff & m \Vdash^S v \in [A_k] \wedge k \in L \\
m \Vdash^S v \in [F[\theta]] & \iff & m \Vdash^S v \in \theta(A^+) \text{ where } F[\Delta] = A^+ \in \Sigma
\end{array}$$

Because we have equirecursive type definitions, the last clause is usually applied silently. The definition of the logical predicate is no longer straightforwardly inductive on the structure of the type. However, we see that for purely positive types (the only ones involved in recursion), the value in the definition becomes strictly smaller in each clause if type definitions are contractive. In other words, we now have a nested inductive definition of the logical predicate, first on the type, and when the type is purely positive, on the structure of the value.

We can also add recursion to our term language with the key proviso that we either restrict ourselves to certain patterns of recursion (for example, primitive recursion), or termination is guaranteed by other external means (for example, an analysis using sized types [2]). This assumption allows us to maintain the structure of the logical predicate, even if it is no longer a means to prove termination (which we are not interested in for this paper).

Lemma 8 (Compositionality (including purely positive equirecursive types)).
Define R_A such that $k\ R_A\ w$ iff $k \Vdash w \in [A]$. Then $m \Vdash^{S,\alpha \mapsto R_A} v \in [B(\alpha)]$ iff $m \Vdash^S v \in [B(A)]$.

Proof.
By nested induction on the definition of the logical predicate for $B(\alpha)$, first on the structure of B and second on the structure of the value when a purely positive type $F[\theta]$ has been reached. ◀

Theorem 9 (Fundamental Theorem (including purely positive recursive types)).
Assume $\Delta \mid \Gamma \,;\, \Omega \vdash e : A$, a mapping S with domain Δ, and two closing substitutions $\epsilon \Vdash^S \theta \in [\Gamma]$ and $m \Vdash^S \eta \in [\Omega]$. Then $m \Vdash^S (\theta \,;\, \eta)(e) \in [\![A]\!]$.

Proof.

By induction on the structure of the given typing derivation. When reasoning about functions and recursion, we need the assumption of termination. ◀

Appendix A Complete definition of the Type System

$$\begin{array}{rrcl} \text{Types} & A, B & ::= & \alpha \mid 1 \mid A \bullet B \mid A \rightarrowtail B \mid A \twoheadrightarrow B \mid \forall \alpha.\, A \\ & & \mid & A \to B \mid A \circ B \mid \oplus\{\ell : A_\ell\}_{\ell \in L} \end{array}$$

$$\begin{array}{rrcl} \text{Purely Positive Types} & A^+, B^+ & ::= & A^+ \bullet B^+ \mid A^+ \circ B^+ \\ & & \mid & \mathbf{1} \mid \oplus\{\ell : A^+_\ell\}_{\ell \in L} \mid F[\theta] \end{array}$$

$$\text{Type Definitions} \quad \Sigma \quad ::= \quad F[\Delta] = A^+ \mid (\cdot) \mid \Sigma_1, \Sigma_2$$

$$\text{Type Substitutions} \quad \theta \quad ::= \quad \alpha \mapsto A^+ \mid (\cdot) \mid \theta_1 \theta_2$$

$$\begin{array}{rrcll} \text{Terms} & e & ::= & x & \\ & & \mid & () \mid \mathbf{match}\ e\ (() \Rightarrow e') & (\mathbf{1}) \\ & & \mid & (e_1, e_2) \mid \mathbf{match}\ e\ ((x, y) \Rightarrow e') & (A \bullet B) \\ & & \mid & \lambda x.\ e \mid e_1\, e_2 & (A \rightarrowtail B, A \twoheadrightarrow B, A \to B) \\ & & \mid & k(e) \mid \mathbf{match}\ e\ \{\ell(x_\ell) \Rightarrow e'\}_{\ell \in L} & (\oplus\{\ell : A_\ell\}) \end{array}$$

Typing rules:

$$\frac{}{\Delta \mid \Gamma ; x : A \vdash x : A}\ \mathsf{hyp}$$

$$\frac{}{\Delta \mid \Gamma ; \cdot \vdash () : \mathbf{1}}\ \mathbf{1}I \qquad \frac{\Delta \mid \Gamma ; \Omega \vdash e : \mathbf{1} \quad \Delta \mid \Gamma ; \Omega_L \Omega_R \vdash e' : C}{\Delta \mid \Gamma ; \Omega_L \Omega \Omega_R \vdash \mathbf{match}\ e\ (() \Rightarrow e') : A}\ \mathbf{1}E$$

$$\frac{\Delta \mid \Gamma ; \Omega\,(x : A) \vdash e : B}{\Delta \mid \Gamma ; \Omega \vdash \lambda x.\, e : A \twoheadrightarrow B} \twoheadrightarrow I \qquad \frac{\Delta \mid \Gamma ; \Omega \vdash e_1 : A \twoheadrightarrow B \quad \Delta \mid \Gamma ; \Omega_A \vdash e_2 : A}{\Delta \mid \Gamma ; \Omega\, \Omega_A \vdash e_1\, e_2 : B} \twoheadrightarrow E$$

$$\frac{\Delta \mid \Gamma ; (x : A)\, \Omega \vdash e : B}{\Delta \mid \Gamma ; \Omega \vdash \lambda x.\, e : A \rightarrowtail B} \rightarrowtail I \qquad \frac{\Delta \mid \Gamma ; \Omega \vdash e_1 : A \rightarrowtail B \quad \Delta \mid \Gamma ; \Omega_A \vdash e_2 : A}{\Delta \mid \Gamma ; \Omega_A\, \Omega \vdash e_1\, e_2 : B} \rightarrowtail E$$

$$\frac{\Delta \mid \Gamma ; \Omega_A \vdash e_1 : A \quad \Delta \mid \Gamma ; \Omega_B \vdash e_2 : B}{\Delta \mid \Gamma ; \Omega_A\, \Omega_B \vdash (e_1, e_2) : A \bullet B} \bullet I$$

$$\frac{\Delta \mid \Gamma ; \Omega \vdash e : A \bullet B \quad \Delta \mid \Gamma ; \Omega_L\,(x : A)\,(y : B)\, \Omega_R \vdash e' : C}{\Delta \mid \Gamma ; \Omega_L\, \Omega\, \Omega_R \vdash \mathbf{match}\ e\ ((x, y) \Rightarrow e') : C} \bullet E$$

$$\frac{\Delta, \alpha\ \mathsf{type} \mid \Gamma ; \Omega \vdash e : A}{\Delta \mid \Gamma ; \Omega \vdash e : \forall \alpha.\, A} \forall I \qquad \frac{\Delta \mid \Gamma ; \Omega \vdash e : \forall \alpha. A(\alpha) \quad \Delta \vdash B\ \mathsf{type}}{\Delta \mid \Gamma ; \Omega \vdash e : A(B)} \forall E$$

$$\frac{}{\Delta \mid \Gamma, x : A ; \cdot \vdash x : A}\ \mathsf{hyp}$$

$$\frac{\Delta \mid \Gamma, x : A ; \Omega \vdash e : B}{\Delta \mid \Gamma ; \Omega \vdash \lambda x.\, e : A \to B} \to I \qquad \frac{\Delta \mid \Gamma ; \Omega \vdash e_1 : A \to B \quad \Delta \mid \Gamma ; \cdot \vdash e_2 : A}{\Delta \mid \Gamma ; \Omega \vdash e_1\, e_2 : B} \to E$$

$$\frac{\Delta \mid \Gamma ; \Omega_A \vdash e_1 : A \quad \Delta \mid \Gamma ; \Omega_B \vdash e_2 : B}{\Delta \mid \Gamma ; \Omega_B\, \Omega_A \vdash (e_1, e_2) : A \circ B} \circ I$$

$$\frac{\Delta \mid \Gamma ; \Omega \vdash e : A \circ B \quad \Delta \mid \Gamma ; \Omega_L\,(y : B)\,(x : A)\, \Omega_R \vdash e' : C}{\Delta \mid \Gamma ; \Omega_L\, \Omega\, \Omega_R \vdash \mathbf{match}\ e\ ((x, y) \Rightarrow e') : C} \circ E$$

$$\frac{(k \in L) \quad \Delta \mid \Gamma ; \Omega \vdash e : A_k}{\Delta \mid \Gamma ; \Omega \vdash k(e) : \oplus\{\ell : A_\ell\}_{\ell \in L}} \oplus I$$

$$\frac{\Delta \mid \Gamma ; \Omega \vdash e : \oplus\{\ell : A_\ell\}_{\ell \in L} \quad (\Delta \mid \Gamma ; \Omega_L\,(x_\ell : A_\ell)\, \Omega_R \vdash e_\ell : A_\ell) \quad (\forall \ell \in L)}{\Delta \mid \Gamma ; \Omega_L\, \Omega\, \Omega_R \vdash \mathbf{match}\ e\ \{\ell(x_\ell) \Rightarrow e_\ell\}_{\ell \in L} : C} \oplus E$$

Operational semantics:

$$\frac{}{() \hookrightarrow ()} \qquad \frac{e \hookrightarrow () \quad e' \hookrightarrow v}{\mathbf{match}\ e\ (() \Rightarrow e') \hookrightarrow v}$$

$$\frac{}{\lambda x.\, e \hookrightarrow \lambda x.\, e} \qquad \frac{e_1 \hookrightarrow \lambda x.\, e_1' \quad e_2 \hookrightarrow v_2 \quad [v_2/x]e_1' \hookrightarrow v}{e_1\, e_2 \hookrightarrow v}$$

$$\frac{e_1 \hookrightarrow v_1 \quad e_2 \hookrightarrow v_2}{(e_1, e_2) \hookrightarrow (v_1, v_2)} \qquad \frac{e \hookrightarrow (v_1, v_2) \quad [v_1/x, v_2/y]e' \hookrightarrow v'}{\mathbf{match}\ e\ ((x, y) \Rightarrow e') \hookrightarrow v'}$$

$$\frac{e \hookrightarrow v}{k(e) \hookrightarrow k(v)} \qquad \frac{e \hookrightarrow k(v) \quad [v/x_k]e_k \hookrightarrow v'}{\mathbf{match}\ e\ \{\ell(x_\ell) \Rightarrow e_\ell\}_{\ell \in L} \hookrightarrow v'}$$

Appendix B Complete definition of the Logical Predicate

$$m \Vdash^S e \in [\![A]\!] \iff \exists v.\, e \hookrightarrow v \wedge m \Vdash^S v \in [A]$$

$$m \Vdash^S v \in [A \bullet B] \iff \exists m_1, m_2.\, m = m_1 \cdot m_2 \wedge v = (v_1, v_2) \wedge m_1 \Vdash^S v_1 \in [A] \wedge m_2 \Vdash^S v_2 \in [B]$$

$$m \Vdash^S v \in [A \twoheadrightarrow B] \iff \forall k.\, k \Vdash^S w \in [A] \Longrightarrow m \cdot k \Vdash^S v\, w \in [\![B]\!]$$

$$m \Vdash^S v \in [A \rightarrowtail B] \iff \forall k.\, k \Vdash^S w \in [A] \Longrightarrow k \cdot m \Vdash^S v\, w \in [\![B]\!]$$

$$m \Vdash^S v \in [\alpha] \iff m\ S(\alpha)\ v$$

$$m \Vdash^S v \in [\forall \alpha.\, A(\alpha)] \iff \forall B, R_B.\, m \Vdash^{S, \alpha \mapsto R_B} v \in [A(\alpha)]$$

$$m \Vdash v \in [A \to B] \iff \forall w.\, \epsilon \Vdash w \in [A] \Longrightarrow m \Vdash v\, w \in [\![B]\!]$$

$$m \Vdash^S v \in [A \circ B] \iff \exists m_1, m_2.\, m = m_2 \cdot m_1 \wedge v = (v_1, v_2) \wedge m_1 \Vdash^S v_1 \in [A] \wedge m_2 \Vdash^S v_2 \in [B]$$

$$m \Vdash^S k(v) \in [\oplus\{\ell : A_\ell\}_{\ell \in L}] \iff m \Vdash^S v \in [A_k] \wedge k \in L$$

$$m \Vdash^S v \in [F[\theta]] \iff m \Vdash^S v \in \theta(A^+) \text{ where } F[\Delta] = A^+ \in \Sigma$$

References

References[1] Andreas Abel and Jean-Philippe Bernardy. A unified view of modalities in type systems. Proceedings of the ACM on Programming Languages, 4(ICFP):1–28, 2020. doi:10.1145/3408972. [2] Andreas Abel and Brigitte Pientka. Well-founded recursion with copatterns and sized types. Journal of Functional Programming, 26:e2, 2016. doi:10.1017/S0956796816000022. [3] Amal Ahmed, Derek Dreyer, and Andreas Rossberg. State-dependent representation independence. In 36th Symposium on Principles of Programming Languages (POPL 2009), pages 340–353, Savannah, Georgia, USA, January 2009. ACM. doi:10.1145/1480881.1480925. [4] Amal Ahmed, Matthew Fluet, and Greg Morrisett. A step-indexed model of substructural state. In 10th International Conference on Functional Programming (ICFP 2005), pages 78–91, 2005. doi:10.1145/1086365.1086376. [5] Amal Ahmed, Matthew Fluet, and Greg Morrisett. L3: a linear language with locations. Fundamenta Informaticae, 77(4):397–449, 2007. [6] Bob Atkey and James Wood. Context constrained computation. In 3rd Workshop on Type-Driven Development (TyDe'18), 2018. [7] Robert Atkey. Syntax and semantics of quantitative type theory. In Anuj Dawar and Erich Grädel, editors, 33rd Conference on Logic in Computer Science (LICS 2018), pages 56–65, Oxford, UK, July 2018. ACM. doi:10.1145/3209108.3209189. [8] Stephanie Balzer, Farzaneh Derakhshan, Robert Harper, and Yue Yao. Logical relations for session-typed concurrency. arXiv preprint arXiv:2309.00192, 2023. doi:10.48550/arXiv.2309.00192. [9] P. N. Benton. A mixed linear and non-linear logic: Proofs, terms and models. In Leszek Pacholski and Jerzy Tiuryn, editors, Selected Papers from the 8th International Workshop on Computer Science Logic (CSL'94), pages 121–135, Kazimierz, Poland, September 1994. Springer LNCS 933. An extended version appears as Technical Report UCAM-CL-TR-352, University of Cambridge. [10] Lars Birkedal, Thomas Dinsdale-Young, Armaël Guéneau, Guilhem Jaber, Kasper Svendsen, and Nikos Tzevelekos. Theorems for free from separation logic specifications. Proceedings of the ACM on Programming Languages, 5(ICFP):1–29, 2021. doi:10.1145/3473586. [11] Luís Caires, Jorge A Pérez, Frank Pfenning, and Bernardo Toninho. Behavioral polymorphism and parametricity in session-based communication. In 22nd European Symposium on Programming (ESOP 2013), pages 330–349, Rome, Italy, March 2013. Springer LNCS 7792. doi:10.1007/978-3-642-37036-6_19. [12] Luís Caires and Frank Pfenning. Session types as intuitionistic linear propositions. In P.Gastin and F.Laroussinie, editors, 21st International Conference on Concurrency Theory (CONCUR 2010), pages 222–236, Paris, France, August 2010. Springer LNCS 6269. doi:10.1007/978-3-642-15375-4_16. [13] Farzaneh Derakhshan, Stephanie Balzer, and Limin Jia. Session logical relations for noninterference. In 36th Annual Symposium on Logic in Computer Science (LICS 2021), pages 1–14. IEEE, 2021. doi:10.1109/LICS52264.2021.9470654. [14] Farzaneh Derakhshan, Stephanie Balzer, and Yue Yao. Regrading policies for flexible information flow control in session-typed concurrency (artifact). Dagstuhl Artifacts Series, 10(2):4–1, 2024. [15] Peng Fu, Kohei Kishida, and Peter Selinger. Linear dependent type theory for quantum programming languages. In 35th Symposium on Logic in Computer Science (LICS 2020), pages 440–453, Saarbrücken, Germany, July 2020. ACM. [16] Simon J. Gay and Vasco T. Vasconcelos. Linear type theory for asynchronous session types. Journal of Functional Programming, 20(1):19–50, January 2010. doi:10.1017/S0956796809990268. [17] Aïna Linn Georges, Benjamin Peters, Laila Albeheiry, Leo White, Stephan Dolan, Richard A. Eisenberg, Chris Casinghino, François Pottier, and Derek Dreyer. Data race freedom à la mode. In Principles of Programming Languages (POPL 2025), volume 9 of Proceedings on Programming Languages, pages 656–686. ACM, January 2025. doi:10.1145/3704859. [18] Jean-Yves Girard and Yves Lafont. Linear logic and lazy computation. In H. Ehrig, R. Kowalski, G. Levi, and U. Montanari, editors, Proceedings of the International Joint Conference on Theory and Practice of Software Development, volume 2, pages 52–66, Pisa, Italy, March 1987. Springer-Verlag LNCS 250. doi:10.1007/BFB0014972. [19] Junyoung Jang, Sophia Roshal, Frank Pfenning, and Brigitte Pientka. Adjoint natural deduction. In Jakob Rehof, editor, 9th International Conference on Formal Structures for Computation and Deduction (FSCD 2024), pages 15:1–15:23, Tallinn, Estonia, July 2024. LIPIcs 299. Extended version available as https://arxiv.org/abs/2402.01428. [20] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. Journal of Functional Programming, 28:e20, 2018. doi:10.1017/S0956796818000151. [21] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. In 42nd Annual Symposium on Principles of Programming Languages (POPL 2015), pages 637–650, Mumbai, India, January 2015. ACM. doi:10.1145/2676726.2676980. [22] Gilles

Kahn. Natural semantics. In Proceedings of the Symposium on Theoretical Aspects of Computer Science, pages 22–39. Springer-Verlag LNCS 247, 1987. doi:10.1007/BFB0039592. [23] Norihiro Kamide. Kripke semantics for modal substructural logics. Journal of Logic, Language and Information, 11:453–470, 2002. doi:10.1023/A:1019915908844. [24] Max Kanovich, Stepan Kuznetsov, Vivek Nigam, and Andre Scedrov. A logical framework with commutative and non-commutative subexponentials. In International Joint Conference on Automated Reasoning (IJCAR 2018), pages 228–245. Springer LNAI 10900, 2018. doi:10.1007/978-3-319-94205-6_16. [25] Joachim Lambek. The mathematics of sentence structure. The American Mathematical Monthly, 65(3):154–170, 1958. [26] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying Rust programs using linear ghost types. In Proceedings of the ACM on Programming Languages, volume 7 (OOPSLA 2023), pages 286–315, April 2023. Extended version available as https://arxiv.org/abs/2303.05491. [27] Anton Lorenzen, Daan Leijen, and Wouter Swierstra. FP2: Fully in-place functional programming. In International Conference on Functional Programming (ICFP 2023), Proceedings on Programming Languages, pages 275–304. ACM, August 2023. doi:10.1145/3607840. [28] Anton Lorenzen, Daan Leijen, Wouter Swierstra, and Sam Lindley. The functional essence of imperative binary search trees. In Programming Language Design and Implementation (PLDI 2024), volume 8 of Proceedings on Programming Languages, pages 518–542. ACM, January 2024. doi:10.1145/3656398. [29] Wendy MacCaull. Relational semantics and a relational proof system for full Lambek calculus. Journal of Symbolic Logic, 63(2):623–637, 1998. doi:10.2307/2586855. [30] John C Mitchell. Representation independence and data abstraction. In Proceedings of the 13th Symposium on Principles of Programming Languages (POPL 1986), pages 263–276. ACM, 1986. doi:10.1145/512644.512669. [31] Aleksandar Nanevski, Frank Pfenning, and Brigitte Pientka. Contextual modal type theory. Transactions on Computational Logic, 9(3), 2008. doi:10.1145/1352582.1352591. [32] Peter W O'Hearn and David J Pym. The logic of bunched implications. Bulletin of Symbolic Logic, 5(2):215–244, 1999. doi:10.2307/421090. [33] Jorge A. Pérez, Luís Caires, Frank Pfenning, and Bernardo Toninho. Termination in session-based concurrency via linear logical relations. In H. Seidl, editor, 22nd European Symposium on Programming (ESOP 2012), pages 539–558, Tallinn, Estonia, March 2012. Springer LNCS 7211. [34] Jorge A. Pérez, Luís Caires, Frank Pfenning, and Bernardo Toninho. Linear logical relations and observational equivalences for session-based concurrency. Information and Computation, 239:254–302, 2014. doi:10.1016/J.IC.2014.08.001. [35] Frank Pfenning and Klaas Pruiksma. Relating message passing and shared memory, proof-theoretically. In S. Jongmans and A. Lopes, editors, 25th International Conference on Coordination Models and Languages (COORDINATION 2023), pages 3–27, Lisbon, Portugal, June 2023. Springer LNCS 13908. Notes to an invited talk. doi:10.1007/978-3-031-35361-1_1. [36] Jeff Polakow. Ordered Linear Logic and Applications. PhD thesis, Department of Computer Science, Carnegie Mellon University, August 2001. [37] Jeff Polakow and Frank Pfenning. Natural deduction for intuitionistic non-commutative linear logic. In J.-Y. Girard, editor, Proceedings of the 4th International Conference on Typed Lambda Calculi and Applications (TLCA'99), pages 295–309, L'Aquila, Italy, April 1999. Springer-Verlag LNCS 1581. doi:10.1007/3-540-48959-2_21. [38] Klaas Pruiksma, Willow Chargin, Frank Pfenning, and Jason Reed. Adjoint logic and its concurrent operational interpretation. Unpublished manuscript, January 2018. [39] Jason Reed. Hybridizing a logical framework. In Proceedings of the International Workshop on Hybrid Logic (HyLo'06), pages 135–148. Electronic Notes in Theoretical Computer Science, v.174(6), 2007. [40] Jason Reed and Frank Pfenning. A constructive approach to the resource semantics of substructural logics. Unpublished Manuscript, May 2010. URL: https://www.cs.cmu.edu/~jcreed/papers/rp-substruct.pdf. [41] Jason C. Reed. A Hybrid Logical Framework. PhD thesis, Carnegie Mellon University, September 2009. Available as Technical Report CMU-CS-09-155. [42] John C. Reynolds. Types, abstraction, and parametric polymorphism. In R.E.A. Mason, editor, Information Processing 83, pages 513–523. Elsevier, September 1983. [43] Christopher Strachey. Fundamental concepts in programming languages. Higher-Order and Symbolic Computation, 13:11–49, 2000. Notes for lecture course given at the International Summer School in Computer Programming at Copenhagen, Denmark, August 1967. doi:10.1023/A:1010000313106. [44] Bas van den Heuvel, Farzaneh Derakhshan, and Stephanie Balzer. Information flow control in cyclic process networks. In 38th European Conference on Object-Oriented Programming (ECOOP 2024), pages 40:1–40:30. LIPIcs 313, 2024. doi:10.4230/LIPICS.ECOOP.2024.40. [45] Victoria Vollmer, Danielle Marshall, Harley Eades, III, and Dominic Orchard. A mixed linear and graded logic: Proofs, terms, and models. In 33rd Conference on Computer Science Logic (CSL 2025), pages 32:1–32:21, Amsterdam, The Netherlands, February 2025. LIPIcs 326. doi:10.4230/LIPICS.CSL.2025.32. [46] Philip Wadler. Theorems for free! In J. Stoy, editor, Proceedings of the 4th International

Conference on Functional Programming Languages and Computer Architecture (FPCA'89), pages 347–359, London, UK, September 1989. ACM. doi:10.1145/99370.99404. [47] Jianzhou Zhao, Qi Zhang, and Steve Zdancewic. Relational parametricity for a polymorphic linear lambda calculus. In K. Ueda, editor, 8th Asian Symposium on Programming Languages and Systems (APLAS 2010), pages 344–359. Springer LNCS 6461, 2010. doi:10.1007/978-3-642-17164-2_24.